Spring Boot 3.X 개발 심화

inky4832@daum.net

5. REST API 개발 심화

- RESTful API 설계 원칙
- Spring MVC 기반 REST Controller
- 요청/응답 처리 심화
- API 문서화와 Swagger

5.1 RESTful API 설계 원칙

REST (Representation State Transfer)

- 개념
  - REST는 웹의 기본 프로토콜인 HTTP를 기반으로 하는 아키텍처 스타일 중 하나
  - 클라이언트와 서버 간의 상호작용을 리소스(Resource)중심으로 설계
  - 상태(State)는 클라이언트에 의해 표현(Representation)
  - RESTful API는 이러한 REST 원칙을 준수하여 만든 웹 기반 데이터 통신 인터페이스임

용어	의미
리소스 (Resource)	서버가 제공하는 대상 ex. 사용자, 게시글, 상품 등
표현 (Representation)	리소스를 주고 받는 방식 ex. JSON, XML 등
상태 (State)	요청 시점의 리소스 상태를 클라이언트가 표현

5.1 RESTful API 설계 원칙

REST 특징

항목	설명
무상태성 (Stateless)	서버는 클라이언트의 상태를 저장하지 않고 들어오는 요청만 단순히 처리 요청 간의 관계는 독립적임 (모든 정보는 매번 요청 필요 ex. 토큰 등)
일관된 인터페이스 (Uniform Interface)	모든 리소스는 동일한 형태의 URI와 HTTP 메서드로 접근함 즉 요청 방식과 응답 구조가 일정함 (개발자 간 협업과 유지보수 용이)
캐시 가능성 (Cacheable)	HTTP 프로토콜의 캐싱 기능을 활용해 응답을 재사용할 수 있음
계층 구조 (Layered System)	서버는 여러 계층(프록시, 로드밸런서, 인증 등)으로 나눌 수 있음 보안과 안정성 향상
자기 서술적 메시지 (Self-descriptive)	요청 및 응답 메시지만으로도 행위를 명확히 이해 가능
클라이언트/서버 구조	UI와 데이터 처리를 명확히 분리하여 확장성과 유지보수성 향상 ex. 백엔드(Spring, Node.js), 프론트엔드(Vue, React)

5.1 RESTful API 설계 원칙

REST 구성 요소

구성요소	설명	예시
리소스 (Resource)	URI로 식별되는 데이터 객체	/api/users, /api/products/1
표현 (Representation)	리소스의 상태를 표현하는 포맷	JSON, XML
HTTP 메서드(Method)	리소스 조작 행위를 명시 (CRUD)	GET, POST, PUT, PATCH, DELETE
상태 코드 (Status Code)	요청 처리 결과를 표현	200,201,400,404,500

5.1 RESTful API 설계 원칙

RESTful 설계 원칙

A. 리소스 중심 설계
- 행위는 HTTP 메서드로 표현하고 URI에는 정보의 자원만을 표현

- 비권장: **/users/show/**1
- 권장: **/users/1**

B. 계층적 구조
- 리소스 간의 관계는 계층형 URI로 표현

/users/{userId}/orders/{orderId}

5.1 RESTful API 설계 원칙

RESTful 설계 원칙

C. 표준 HTTP 메서드 사용
- 리소스에 대한 조작 행위는 HTTP 메서드 이용

메서드	의미	행위
GET	조회	리소스 읽기
POST	생성	새로운 리소스 추가
PUT	수정	전체 리소스 교체
PATCH	부분 수정	리소스 일부 갱신
DELETE	삭제	리소스 제거

5.1 RESTful API 설계 원칙

RESTful 설계 원칙

D. 상태 코드 명확화

- REST는 상태 코드 값을 이용하여 요청에 대한 결과를 표현

상태 코드	의미	설명
200 OK	성공	요청이 정상 처리됨
201 Created	생성 성공	POST / PUT 요청으로 자원 생성됨
400 Bad Request	요청 오류	요청 형식 또는 값이 잘못됨
401 Unauthorized	미승인	인증 필요
403 Forbidden	권한 없음	권한 필요
404 Not Found	리소스 없음	잘못된 URI
405 Method Not Allowed	요청 메서드 미지원	메서드 지원 필요
500 Internal Server Error	서버 오류	서버 내부 문제 발생

5.1 RESTful API 설계 원칙

RESTful 설계 원칙

E. 일관된 응답 구조

- REST API 응답은 일관성 있는 JSON 구조를 항상 유지

```
{
 "status": 200,
 "message": "SUCCESS",
 "data": { "id": 1, "name": "홍길동" }
}
```

5.1 RESTful API 설계 원칙

URI 설계 규칙

항목	규칙	예시
리소스명	복수형 명사 사용	/users, /products
행위 표현 금지	HTTP 메서드에 의한 행위 표현	/users/1 (권장) /users/show/1 (비권장)
대소문자	소문자 사용	/users (권장) /Users (비권장)
구분자	단어 구분 시 하이픈(-) 사용	/user-orders
필터링/검색	쿼리 파라미터 활용 URL 파라미터 활용	/users?age=20&sort=desc /users/20/desc

5.2 Spring MVC 기반 REST Controller

@RestController

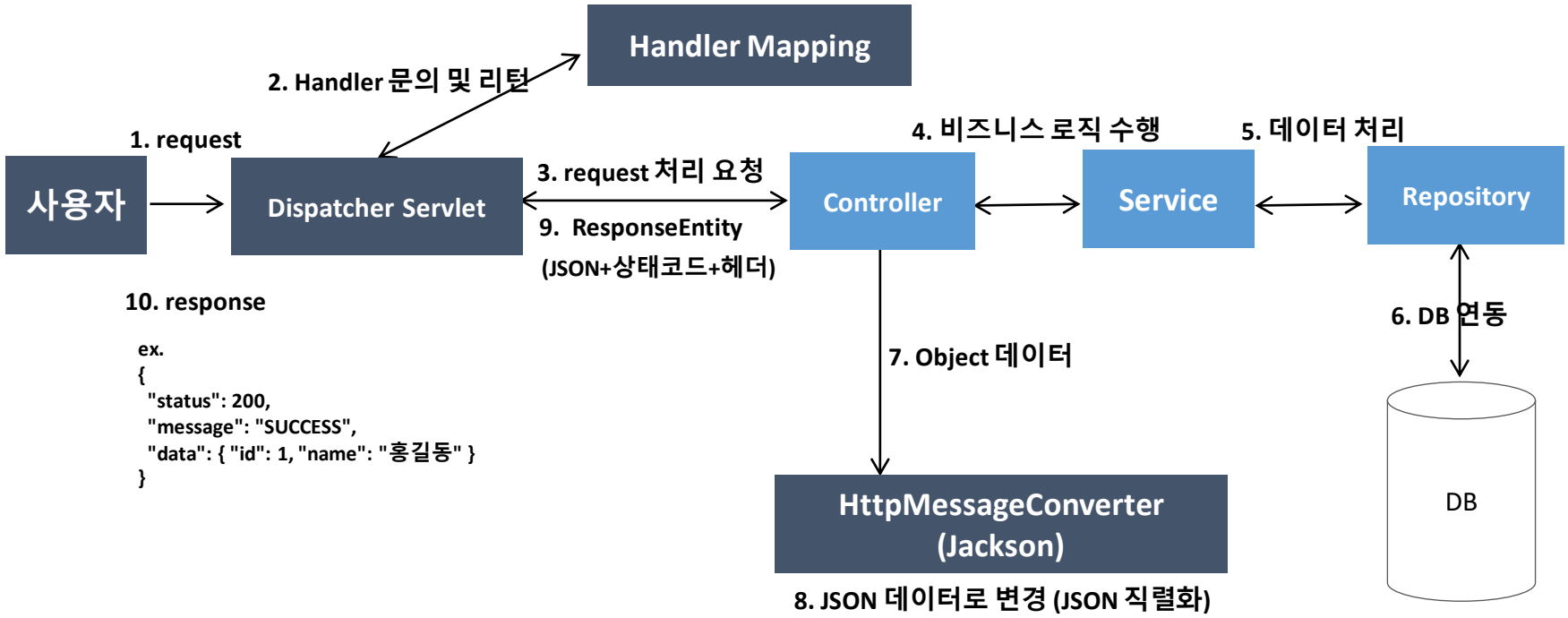
- 개념
  - Spring MVC의 @Controller에 @ResponseBody가 결합된 형태로서 View 없이 JSON/XML 형태의 데이터만 반환하는 RESTful 전용 컨트롤러임
  - 즉, 화면 랜더링 대신 데이터 응답(API Response)에 초점을 맞춘 구조임

- 주요 구성 요소

구성 요소	역할	어노테이션
Dispatcher Servlet	요청을 중앙에서 받아 컨트롤러로 분배	자동 등록
Controller	요청 처리 및 서비스 호출	@RestController, @RequestBody
Service	비즈니스 로직 및 트랜잭션 처리	@Service
Repository	DB 접근 계층 (CRUD 수행)	@Repository, JpaRepository
ResponseEntity	응답 데이터(JSON) + 상태코드 + 헤더 반환	ResponseEntity

5.2 Spring MVC 기반 REST Controller

@RestController 요청 흐름



5.2 Spring MVC 기반 REST Controller

@RestController 특징

항목	설명
핵심 역할	JSON/XML 형태로 데이터를 반환하는 REST 전용 컨트롤러 @Controller + @ResponseBody = @RestController
View 반환 없음	JSP/Thymeleaf 등 뷰 랜더링 대신에 JSON 형태로 반환
자동 JSON 변환	Jackson 라이브러리를 이용하여 자바 객체를 JSON으로 자동 변환
HTTP 상태코드 제어	ResponseEentity 사용 시 명시적인 상태코드 제어 가능
요청 바인딩 간소화	@RequestBody, @PathVariable, @RequestParam 등으로 요청 데이터 손쉽게 매핑

5.2 Spring MVC 기반 REST Controller

@RestController vs @Controller

비교 항목	@Controller	@RestController
주요 목적	HTML 랜더링 (SSR)	JSON/XML 데이터 응답 (API)
Content-Type	text/html	application/json
응답 포맷	HTML (뷰)	JSON (데이터)
클라이언트 역할	단순 랜더링 담당	랜더링 및 로직 담당
ViewResolver 사용	사용함 (JSP/Thymeleaf)	사용하지 않음
HttpMessageConverter	사용하지 않음	사용함 (Jackson, JSON 직렬화)
구성 요소	Controller + View	Controller + JSON데이터
대표 프레임워크 연계	JSP, Thymeleaf	React, Vue, Angular 등

5.2 Spring MVC 기반 REST Controller

주요 어노테이션

어노테이션	주요 기능	주 사용 HTTP Method	특징
@RequestMapping	URL과 컨트롤러 연결	모든 메서드	공통 경로 설정
@GetMapping	데이터 조회	GET	조회 전용
@PostMapping	데이터 생성	POST	요청 본문(body)필요
@PutMapping	데이터 전체 수정	PUT	기존 리소스 대체
@PatchMapping	데이터 부분 수정	PATCH	부분 리소스 대체
@DeleteMapping	데이터 삭제	DELETE	리소스 제거
@RequestBody	요청본문(JSON)을 자바객체로 변환	POST/PUT	Jackson 자동 변환
@ResponseBody	자바객체를 JSON으로 변환	모든 메서드	@RestController 내장
@PathVariable	URL 파라미터 값 추출	모든 메서드	/users/{id}
@RequestParam	쿼리 파라미터 추출	GET	/users?key=value

5.2 Spring MVC 기반 REST Controller

@RestController 예시

```
@RestController
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @GetMapping("/hello-world")
    public String helloWorld() {
            return "Hello World";
    }

    @GetMapping("/users/{id}")
    public User retrieveUser(@PathVariable int id) {
            return userService.findById(id);
    }
    @GetMapping("/users")
    public List<User> retrieveAllUsers() {
            return userService.findAll();
    }

    @PostMapping("/users")
    public void createUser(@RequestBody User user) {
            userService.save(user);
    }
    @PutMapping("/users/{id}")
    public void updateUser(@PathVariable int id, @RequestBody User user) {
            userService.updateUser(id, user);
    }
    @DeleteMapping("/users/{id}")
    public void deleteUser(@PathVariable int id) {
            userService.deleteById(id);
    }
}
```

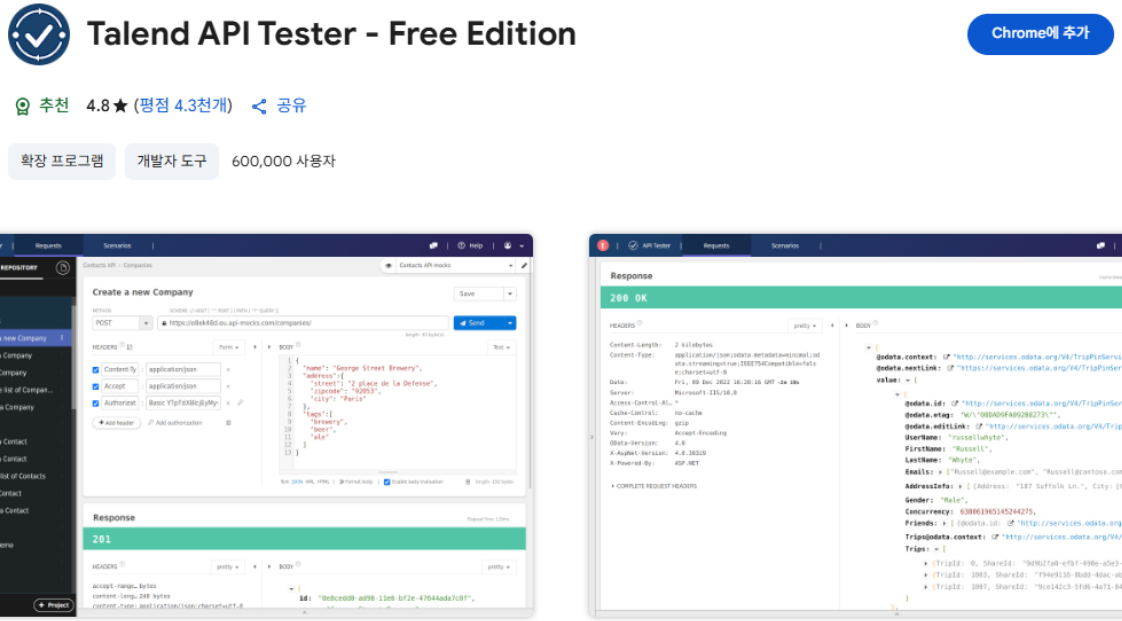
5.2 Spring MVC 기반 REST Controller

REST API 테스트 도구

- 용도
  - POST/PUT/DELETE 요청은 기본 웹 브라우저에서 직접 요청이 불가능
  - 따라서 REST API 요청을 대신 해주는 테스트 도구가 필요
- 종류
  - Postman
  - Talend API Tester (크롬 플러그인)
  - curl

5.2 Spring MVC 기반 REST Controller

Talend API Tester 설치



5.2 Spring MVC 기반 REST Controller

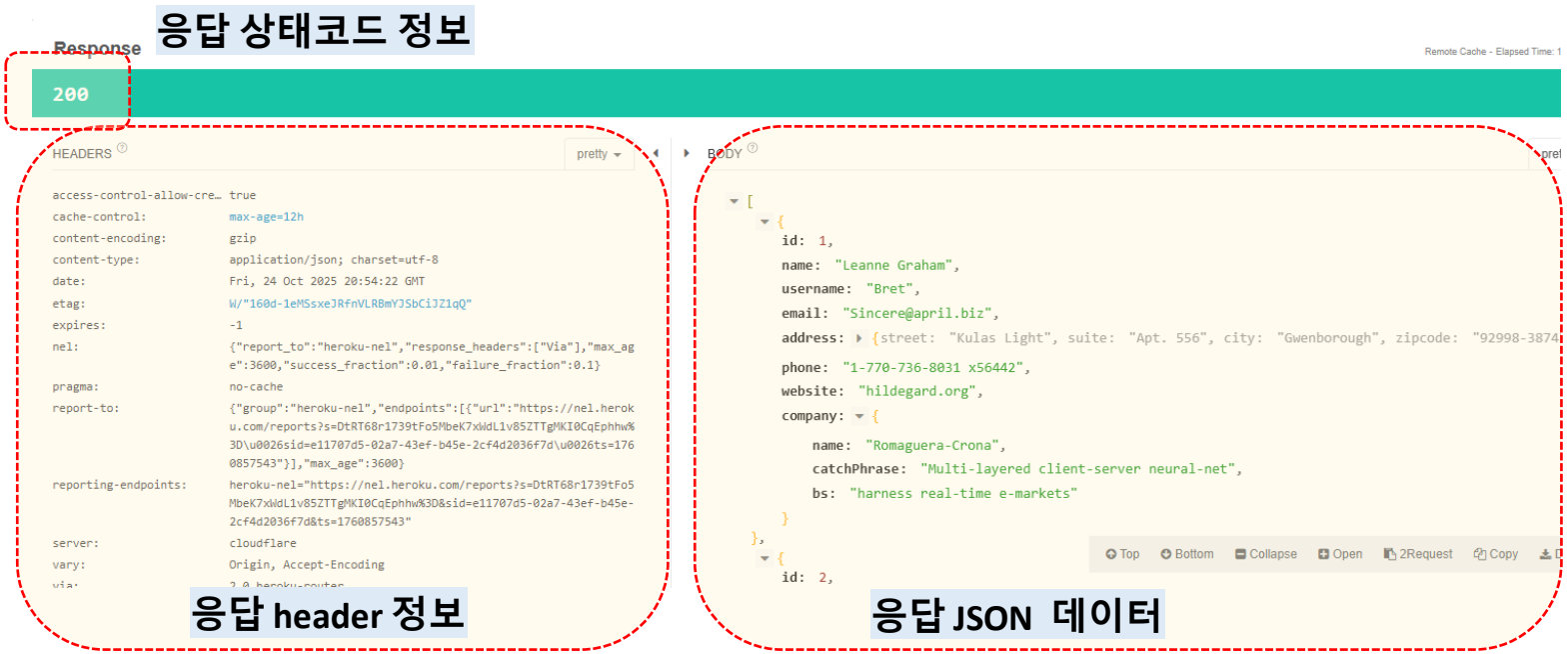
Talend API Tester 실행 및 요청

샘플 URL : GET https://jsonplaceholder.typicode.com/users



5.2 Spring MVC 기반 REST Controller

Talend API Tester 실행결과



5.3 요청/응답 처리 심화

요청/응답 처리 심화

- 개념
  - REST API는 클라이언트의 요청(Request)과 서버의 응답(Response)으로 구성됨
  - 이때 단순히 데이터를 주고 받는 것이 아니라, 데이터 검증, 변환, 예외처리, 표준화된 응답 구조, 직렬화/역직렬화 등을 포함해야 안정적인 API가 됨

- 지원 API

API	설명
ApiResponse /ResponseEntity	모든 응답을 동일한 JSON 포맷으로 관리 상태 코드, 본문, 헤더를 통합적으로 관리
@RestControllerAdvice	전역 예외 처리 클래스 활용한 예외 응답의 일관성 유지
MessageConverter	JSON과 객체 상호 간에 자동 변환 (Jackson) @ResponseBody와 @RequestBody 지원
Validation	@Validated + @RestControllerAdvice 로 입력 값 검증 및 오류 응답 일원화

5.3 요청/응답 처리 심화

ApiResponse

- 기존 문제점
  - 컨트롤러에서의 응답 형식이 제각각 ex. UserDTO, String, void 등
  - 클라이언트 입장에서는 응답 데이터 구조가 일관되지 않아서 파싱/에러처리 어려움
  - 예외 발생시 JSON 구조 불명확 등
  - 위 문제점을 해결하기 위해서 모든 응답을 동일한 JSON 포맷으로 구성

- 목적
  - 모든 API 응답을 단일 JSON 구조로 통일
  - 성공 / 실패를 명확히 구분
  - 상태코드와 메시지, 데이터 포함
  - 프론트엔드 / 외부 연동 시 일관성 있는 파싱 가능

5.3 요청/응답 처리 심화

ApiResponse

- 응답 표준 구조 예시

```
{
  "success": true,
  "message": "요청이 성공했습니다",
  "data": {
    "id": 1,
    "name": "홍길동",
    "email": "hong@test.com"
    }
 }
```

5.3 요청/응답 처리 심화

ApiResponse 코드 예

```java
// ApiResponse.java
package com.example.demo.api;

import lombok.*;

@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class ApiResponse<T> {
    private boolean success;
    private String message;
    private T data;

    public static <T> ApiResponse<T> ok(String message, T data) {
        return ApiResponse.<T>builder()
                .success(true)
                .message(message)
                .data(data)
                .build();
    }

    public static <T> ApiResponse<T> fail(String message) {
          return ApiResponse.<T>builder()
                  .success(false)
                  .message(message)
                  .build();
      }
  }
```

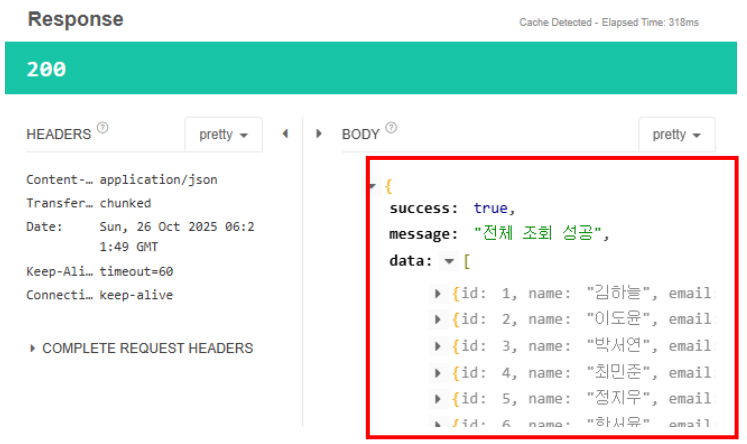
5.3 요청/응답 처리 심화

샘플예– 모든 사용자 목록 보기

개선 전



개선 후



5.3 요청/응답 처리 심화

ResponseEntity

- 개념
  - Spring MVC의 HTTP 응답을 객체 수준으로 통제 할 수 있게 해주는 클래스
  - 단순히 데이터를 응답하는 대신 다음 세 요소를 명시적으로 설정 가능
  - 클라이언트 입장에서는 요청에 대한 명확한 응답상태를 제공 받을 수 있음

- 설정 요소

요소	설명
상태코드 (Status Code)	HTTP 상태코드 ex. 200, 201, 400, 401, 403, 404, 405, 500 HttpStatus에서 상수로 지원 ex. HttpStatus.NOT_FOUND
헤더 (Header)	Authorization, Cache-Control, X-Correlation-Id, Location 등 메타 데이터 HttpHeaders 이용
본문 (Body)	JSON, XML, 문자열, 파일, 표준 응답 객체 등 컨텐츠 본문 ApiResponse 같은 표준화된 응답 객체 권장

5.3 요청/응답 처리 심화

ResponseEntity 주요 메서드

메서드	의미/상태	리턴타입	비고/예시
ok() ok(T)	200 OK 200 OK + 본문	BodyBuilder ResponseEntity<T>	ResponseEntity.ok().body(obj) ResponseEntity.ok(obj)
created(URI location)	201 Created + location	BodyBuilder	ResponseEntity.created(URI.create("/api/users/1")).build()
noContent()	204 No Content	HeaderBuilder	삭제/갱신 후 본문 없음
badRequest()	400 Bad Request	BodyBuilder	입력 오류 시
notFound()	404 Not Found	HeaderBuilder	리소스 없음
status(HttpStatus status) status(int status)	임의 상태 지정 임의 상태 지정(숫자)	BodyBuilder	ResponseEntity.status(HttpStatus.OK) ResponseEntity.status(200)

5.3 요청/응답 처리 심화

빌더(BodyBuilder/HeaderBuilder) 주요 메서드

메서드	용도	예시
header(name, …values) headers(HttpHeaders headers)	임의 헤더 추가 헤더 일괄 설정	.header("X-Correlation-Id", UUID.randomUUID().toString()) .headers(httpHeaders)
contenType(MediaType)	Content-Type 지정	.contentType(MediaType.APPLICATION_JSON)
contentLengh(long)	Content-Length지정	.contentLength(123L)
location(URI)	Location 지정	.location(URI.create("/api/users/1"))
body(T body)	본문설정 + 빌드	.body(dto)
build()	본문없이 빌드	.build()

5.3 요청/응답 처리 심화

인스턴스 getter 주요 메서드

메서드	반환	설명
getStatusCode()	HttpStatusCode	상태 코드 객체
getStatusCodeValue()	int	숫자 코드 ex. 200
getHeaders()	HttpHeaders	헤더
getBody()	T	본문

5.3 요청/응답 처리 심화

ResponseEntity 기본 예시

```
// 다음은 201(Created) 응답코드 반환
@PostMapping("/users")
public ResponseEntity<UserDTO> createUser(@RequestBody UserDTO dto) {
    userService.create(dto);
    return ResponseEntity.created(null).build();
}

// 다음은 201(Created) 응답코드와 header에 location 정보 제공함.
@PostMapping("/users")
public ResponseEntity< UserDTO > createUser(@RequestBody UserDTO dto) {
    UserDTO savedUser  = userService. create(dto);
    URI location = ServletUriComponentsBuilder.fromCurrentRequest().path("/{id}")
                    .buildAndExpand(savedUser.getId())    //위의 {id}를 치환함
                      .toUri();
    return ResponseEntity.created(location).build();
}
```

5.3 요청/응답 처리 심화

ResponseEntity + ApiResponse 기본 예시

```
 /** 단건 조회 */
@GetMapping("/users/{id}")
public ResponseEntity<ApiResponse<UserDTO>> retrieveUser(@PathVariable("id") Long id) {
    UserDTO user = userService.findById(id);
    if (user == null) {
      return ResponseEntity.status(HttpStatus.NOT_FOUND)
                    .body(ApiResponse.error("User not found"));
    }
    return ResponseEntity.ok(ApiResponse.ok("User retrieved successfully", user));
}

/** 전체 조회 */
@GetMapping("/users")
public ResponseEntity<ApiResponse<List<UserDTO>>> retrieveAllUsers() {
    List<UserDTO> users = userService.findAll();
    return ResponseEntity.ok(ApiResponse.ok("All users retrieved", users));
}
```

5.3 요청/응답 처리 심화

전역 예외처리

- 개념
  - Controller 내부 로직의 로컬 예외 처리를 전역적으로 예외처리가 가능하도록 지원

- 필요 API

기술	설명
사용자 정의 예외클래스	extends RuntimeException
@ResponseStatus	특정 예외가 발생될 때 반환할 HTTP 상태코드 선언
@ExceptionHandler	발생된 예외타입을 가로채 사용자 정의 응답으로 변환
@RestControllerAdvice	REST 기반 전역 예외처리 담당

5.3 요청/응답 처리 심화

샘플예(@RestControllerAdvice)

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    /** 404 - EntityNotFoundException */
    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<ApiResponse<Void>> handleNotFound(
            EntityNotFoundException ex, HttpServletRequest req) {
        ApiResponse<Void> body = ApiResponse.of(HttpStatus.NOT_FOUND, ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    /** 409 - 이메일 중복 */
    @ExceptionHandler(DuplicateEmailException.class)
    public ResponseEntity<ApiResponse<Void>> handleDuplicateEmail(
            DuplicateEmailException ex, HttpServletRequest req) {

        ApiResponse<Void> body = ApiResponse.of(HttpStatus.CONFLICT, ex.getMessage());
        return ResponseEntity.status(HttpStatus.CONFLICT).body(body);
    }
}
```

수고하셨습니다.